D O C U M E N T A C I Ó N T É C N I C A

Frontend React — Proyecto **Vitrio**

Dashboard SPA para Sistema de Digital Signage

Samuel Ibáñez Rincón
Andrés Camilo Plazas
David Alejandro Cañon
Manuel Alejandro Betancurt
Diego Florez Rojas

1. Arquitectura del Frontend

El frontend de Vitrio es una Single Page Application (SPA) construida con React 19 y empaquetada con Vite 8. Toda la lógica de presentación, autenticación y consumo de API reside en un único archivo principal App.jsx, lo que simplifica el despliegue y la colaboración en equipo.

Capa	Tecnología / Archivo	Rol
Empaquetador	Vite 8	Servidor de desarrollo y build de producción
UI Framework	React 19 (Hooks)	Componentes funcionales con estado reactivo
Punto de entrada	src/main.jsx	Monta la app en el DOM con StrictMode
Lógica principal	src/App.jsx	Todos los componentes, esquemas y llamadas a la API
Estilos	Inline styles (CSS-in-JS)	Paleta navy/azul, tema oscuro sin dependencias externas
Comunicación API	Fetch nativo + JWT Bearer	Peticiones HTTP autenticadas al backend Django
Persistencia local	localStorage	Almacena los tokens JWT entre sesiones
Túnel desarrollo	ngrok	Expone el backend Django a Internet durante el desarrollo

2. Stack Tecnológico y Dependencias

El proyecto mantiene un stack intencionalmente minimalista: React y React-DOM son las únicas dependencias de producción. No se usan librerías de componentes externas (como MUI o Ant Design), lo que reduce el tamaño del bundle y simplifica el mantenimiento.

2.1 Dependencias de producción

Paquete	Versión	Función
react	^19.2.4	Librería principal para construir la interfaz de usuario
react-dom	^19.2.4	Renderizado de componentes React en el DOM del navegador

2.2 Dependencias de desarrollo

Paquete	Versión	Función
vite	^8.0.1	Empaquetador ultrarrápido con HMR (Hot Module Replacement)
@vitejs/plugin-react	^6.0.1	Plugin oficial de Vite para soporte de JSX y Fast Refresh
eslint	^9.39.4	Linter para detectar errores y malas prácticas en el código
eslint-plugin-react-hooks	^7.0.1	Valida el uso correcto de los Hooks de React
@types/react	^19.2.14	Tipos TypeScript para autocompletado en editores

2.3 Scripts disponibles

```
npm run dev        →  Inicia el servidor de desarrollo en http://localhost:5173
npm run build      →  Genera el bundle optimizado en /dist
npm run preview    →  Previsualiza el build de producción localmente
npm run lint       →  Ejecuta ESLint sobre todos los archivos fuente
```

3. Estructura del Proyecto

El proyecto sigue la estructura estándar de Vite. Al ser una aplicación académica, la lógica está centralizada en App.jsx para facilitar la colaboración y la presentación del código.

```
vitrio-app/
├── index.html              →  HTML raíz; punto de entrada de Vite
├── vite.config.js          →  Configuración de Vite con plugin React
├── eslint.config.js        →  Reglas de linting
├── package.json            →  Metadatos y dependencias del proyecto
├── public/
│   ├── favicon.svg         →  Ícono del sitio
│   └── icons.svg           →  Sprites SVG
└── src/
    ├── main.jsx            →  Punto de entrada React (StrictMode + createRoot)
    ├── App.jsx             →  Toda la lógica: componentes, schemas, auth, API
    ├── App.css             →  Estilos globales mínimos
    └── index.css           →  Reset CSS
```

4. Sistema de Autenticación JWT

El frontend implementa autenticación basada en JSON Web Tokens (JWT). El flujo es stateless: el backend emite dos tokens en el login y el frontend los rota automáticamente antes de cada petición a la API.

4.1 Almacenamiento de tokens

Los tokens se guardan en localStorage del navegador usando claves prefijadas con 'vitrio_':

vitrio_access	Token de acceso (duración: 8 horas, configurado en backend)
vitrio_refresh	Token de refresco (duración: 30 días, para obtener nuevo access)

4.2 Funciones de gestión de tokens

Función	Firma	Descripción
getAccess()	() → string | null	Lee el access token de localStorage
getRefresh()	() → string | null	Lee el refresh token de localStorage
saveTokens()	(access, refresh) → void	Guarda ambos tokens tras el login exitoso
clearTokens()	() → void	Elimina ambos tokens al cerrar sesión
refreshAccessToken()	async () → string | null	Rota el access token usando el refresh token

4.3 Flujo de autenticación

Cada vez que el usuario realiza una acción que requiere la API, el sistema ejecuta el siguiente ciclo:

- 1. Se llama a refreshAccessToken() — hace POST /api/token/refresh/ con el refresh token
- 2. Si la rotación es exitosa, se actualiza el access token en localStorage
- 3. Se ejecuta la petición original con el nuevo token en el header Authorization: Bearer {token}
- 4. Si el servidor responde 401 o 403, se fuerza el logout y se redirige al login

4.4 Header especial de ngrok

Durante el desarrollo, el backend corre bajo un túnel ngrok que muestra una página de advertencia en el navegador. Para evitarla, todas las peticiones incluyen el siguiente header adicional:

```
"ngrok-skip-browser-warning": "true"
```

Este header es ignorado en producción y solo tiene efecto mientras el backend esté expuesto vía ngrok.

5. Capa de Comunicación con la API

Toda la comunicación HTTP pasa por la función centralizada apiFetch. Esto garantiza que cada petición incluya los headers correctos, el token JWT y el manejo uniforme de errores.

5.1 Constante API_BASE

```
const API_BASE = "https://presymphonic-nonnavigably-rhys.ngrok-free.dev";
```

URL base del backend Django. En producción debe reemplazarse por el dominio definitivo del servidor.

5.2 Función apiFetch

Función asíncrona que encapsula todas las llamadas HTTP al backend:

Parámetro	Tipo	Descripción
path	string	Ruta relativa del endpoint (ej: /api/clientes/)
method	string	Método HTTP: GET, POST, PUT, DELETE (default: GET)
body	object | null	Cuerpo de la petición serializado como JSON (default: null)

5.3 Endpoints consumidos

Método	Endpoint	Descripción
POST	/api/login/	Login: recibe username + password, retorna access + refresh tokens
POST	/api/token/refresh/	Rotación de token: recibe refresh, retorna nuevo access token
GET	/api/{entidad}/	Lista todos los registros de una entidad
POST	/api/{entidad}/	Crea un nuevo registro de la entidad
PUT	/api/{entidad}/{id}/	Actualiza completamente un registro existente

Método	Endpoint	Descripción
DELETE	/api/{entidad}/{id}/	Elimina un registro por su ID

6. Esquemas de Datos (SCHEMAS)

El objeto SCHEMAS define la forma de cada entidad desde el punto de vista del frontend. Es la fuente única de verdad para generar los formularios y las columnas de las tablas de manera dinámica. Un cambio en SCHEMAS se refleja automáticamente en toda la interfaz sin modificar los componentes.

6.1 Clientes

Campo (key)	Etiqueta	Tipo de campo
`nombre`	Nombre	text
`email`	Email	email
`telefono`	Teléfono	text

6.2 Usuarios

Campo (key)	Etiqueta	Tipo de campo
`fecha_nacimiento`	Fecha Nacimiento	date
`rol`	Rol	select → ADMINISTRADOR | ADMINISTRADOR_CLIENTE | CLIENTE
`id_cliente`	ID Cliente	number (FK → Cliente)

6.3 Proyectos

Campo (key)	Etiqueta	Tipo de campo
`nombre`	Nombre	text
`descripcion`	Descripción	textarea
`fecha_inicio`	Fecha Inicio	date
`fecha_entrega`	Fecha Entrega	date
`estado`	Estado	select → PROPUESTO | ANALISIS | APROBADO | ... (10 opciones)
`id_cliente`	ID Cliente	number (FK → Cliente)

6.4 Plantillas

Campo (key)	Etiqueta	Tipo de campo
`nombre`	Nombre	text
`descripcion`	Descripción	textarea
`tipo`	Tipo	select → VIDEO | IMAGEN | AUDIO | PAQUETE | METADATO | DOCUMENTO
`version`	Versión	text
`estado`	Estado	select → ACTIVA | INACTIVA | DEPRECADA
`ruta_base`	Ruta Base	text

6.5 Archivos

Campo (key)	Etiqueta	Tipo de campo
`nombre`	Nombre	text
`tipo`	Tipo	select → VIDEO | IMAGEN | AUDIO | PAQUETE | METADATO | DOCUMENTO
`ruta`	Ruta	text
`id_proyecto`	ID Proyecto	number (FK → Proyecto)
`id_plantilla`	ID Plantilla	number (FK → Plantilla)

6.6 Dispositivos

Campo (key)	Etiqueta	Tipo de campo
`nombre`	Nombre	text
`ubicacion`	Ubicación	text
`resolucion`	Resolución	text
`estado`	Estado	select → ACTIVO | INACTIVO | SIN_CONEXION
`id_proyecto`	ID Proyecto	number (FK → Proyecto)

7. Componentes de la Interfaz

La aplicación está compuesta por 7 componentes funcionales de React, todos definidos en App.jsx. Los componentes se comunican mediante props y callbacks; no se usa ninguna librería de gestión de estado global (como Redux o Zustand).

7.1 App — Componente Raíz

Componente exportado por defecto. Actúa como enrutador de primer nivel: decide si mostrar el login o el dashboard según el estado de autenticación.

Estado	Tipo	Descripción
logged	boolean	true si el usuario está autenticado, false si no lo está
checking	boolean	true mientras verifica si existe un token en localStorage al cargar

Al montar, lee el access token de localStorage. Si existe, asume la sesión activa sin hacer una llamada a la API. Renderiza un splash screen ('Cargando...') mientras dura esta verificación.

7.2 LoginScreen — Pantalla de Autenticación

Pantalla completa que recibe el prop onLogin (callback). Presenta el formulario de credenciales y gestiona el flujo de login.

Estado	Tipo	Descripción
user	string	Valor del campo de nombre de usuario (inicializado como 'manuel')
pass	string	Valor del campo de contraseña (inicializado como 'front123')
show	boolean	Controla la visibilidad de la contraseña (toggle ojo)
loading	boolean	Desactiva el botón y muestra 'Ingresando...' durante el login
error	string	Mensaje de error mostrado en el banner rojo si el login falla

- Soporta login con tecla Enter en ambos campos (onKeyDown)
- POST a /api/login/ con username y password
- En caso de error muestra un banner rojo con el mensaje correspondiente
- En caso de éxito llama a saveTokens() y ejecuta el callback onLogin

7.3 Dashboard — Panel Principal

Componente principal post-autenticación. Integra el sidebar de navegación, las tarjetas de estadísticas, la tabla de datos y los modales. Es el orquestador de toda la lógica CRUD.

Estado	Tipo	Descripción
active	string	Entidad actualmente seleccionada (ej: 'clientes', 'proyectos')
data	object	Caché local de todos los registros por entidad { clientes: [], ... }
loading	boolean	Muestra el indicador de carga mientras se consulta la API
modal	string | null	'create', 'edit', 'delete' o null si no hay modal abierto
selected	object | null	Registro seleccionado para editar o eliminar
toast	object | null	Notificación temporal { msg, type } mostrada en la esquina
search	string	Texto del buscador para filtrar registros en tiempo real

La función load(entity) carga los datos de una entidad desde la API y actualiza el caché local. Se envuelve en useCallback para evitar recreaciones innecesarias y se ejecuta automáticamente con useEffect cada vez que cambia la entidad activa.

7.4 EntityTable — Tabla de Registros

Componente de tabla genérico que renderiza cualquier entidad. Lee el schema de la entidad para determinar las columnas dinámicamente.

Prop	Tipo	Descripción
entity	string	Clave de la entidad (ej: 'clientes'). Determina columnas y metadatos
data	array	Array de objetos a mostrar en las filas
onEdit	function	Callback llamado con el objeto de la fila al pulsar 'Editar'
onDelete	function	Callback llamado con el objeto de la fila al pulsar 'Eliminar'

- Los campos estado, rol y tipo se renderizan como badges de colores (ESTADO_COLORS)
- Los emails se resaltan en azul claro para distinguirlos visualmente
- Cada fila tiene efecto hover con fondo azul traslúcido via onMouseEnter/Leave
- Si data está vacío, muestra un estado vacío con el ícono de la entidad

- El índice se muestra con padding de ceros (01, 02, 03...)

7.5 EntityForm — Formulario Genérico

Formulario de creación y edición generado dinámicamente a partir del SCHEMA de la entidad activa. Un solo componente sirve para crear y editar cualquier entidad.

Prop	Tipo	Descripción
entity	string	Entidad para leer su schema y saber qué campos renderizar
initial	object | null	Si se pasa, pre-rellena el formulario (modo edición)
onSubmit	function	Callback con el objeto del formulario al guardar
onCancel	function	Callback para cerrar el modal sin guardar

El tipo de campo en el schema determina qué input se renderiza:

text / email / number / date	Renderiza un <input> estándar con type correspondiente
textarea	Renderiza un <textarea> con resize vertical habilitado
select	Renderiza un <select> con las opciones definidas en el schema

7.6 Modal — Contenedor de Diálogos

Componente de overlay reutilizable con backdrop blur. Contiene cualquier contenido (formularios o confirmaciones) y tiene un botón de cierre en el encabezado. Usa position: fixed y z-index: 1000.

Prop	Tipo	Descripción
title	string	Texto del encabezado del modal
onClose	function	Callback ejecutado al pulsar el botón ✕
children	ReactNode	Contenido interno (formulario, confirmación, etc.)

7.7 ConfirmDialog — Diálogo de Confirmación

Modal especializado para confirmar la eliminación de registros. Muestra un mensaje de advertencia con dos botones: Eliminar (rojo) y Cancelar.

Prop	Tipo	Descripción
onConfirm	function	Callback ejecutado al confirmar la eliminación
onCancel	function	Callback ejecutado al cancelar la acción

7.8 VitrioLogo — Logo SVG

Componente que renderiza el logotipo de Vitrio como SVG inline con gradiente lineal (azul claro → azul oscuro). Acepta un prop size para controlar las dimensiones. Aparece en el sidebar y en la pantalla de login.

8. Flujo Completo de una Operación CRUD

El siguiente diagrama describe el ciclo de vida completo de cualquier operación (Crear, Leer, Actualizar, Eliminar) desde la interacción del usuario hasta la actualización de la interfaz.

8.1 Leer (GET — Cargar datos)

- Usuario selecciona una entidad en el sidebar o cambia la entidad activa
- useEffect dispara load(entity)
- apiFetch llama GET /api/{entity}/ con token JWT
- La respuesta (array) se guarda en el estado data[entity]
- EntityTable re-renderiza con los nuevos datos

8.2 Crear (POST)

- Usuario pulsa '+ Nuevo {Entidad}'
- Se abre Modal con EntityForm vacío (initial=null)
- Usuario completa el formulario y pulsa 'Crear registro'
- handleCreate() llama apiFetch POST /api/{entity}/ con los datos del form
- En éxito: cierra modal, recarga datos, muestra toast verde 'Registro creado ✓'
- En error: muestra toast rojo 'Error al crear registro'

8.3 Editar (PUT)

- Usuario pulsa 'Editar' en una fila de la tabla
- El registro seleccionado se guarda en el estado selected
- Se abre Modal con EntityForm pre-rellenado (initial=selected)
- Usuario modifica campos y pulsa 'Guardar cambios'
- handleEdit() llama apiFetch PUT /api/{entity}/{id}/ con el form actualizado
- En éxito: cierra modal, recarga datos, muestra toast verde 'Cambios guardados ✓'

8.4 Eliminar (DELETE)

- Usuario pulsa 'Eliminar' en una fila → se guarda selected
- Se abre ConfirmDialog
- Usuario confirma → handleDelete() llama apiFetch DELETE /api/{entity}/{id}/
- En éxito: cierra modal, recarga datos, muestra toast rojo 'Registro eliminado'

8.5 Búsqueda en tiempo real

El buscador filtra los registros ya cargados en memoria (sin llamadas adicionales a la API). El filtro aplica sobre todos los valores del objeto usando String().toLowerCase() e includes():

```
filtered = data[active].filter(row =>
  Object.values(row).some(v =>
    String(v).toLowerCase().includes(search.toLowerCase())
  )
)
```

9. Sistema de Diseño Visual

El frontend usa una paleta oscura (dark theme) con azules como color de acento, implementada con estilos inline en JSX. No se usa ningún framework de CSS externo.

9.1 Paleta de colores principal

Variable conceptual	Valor HEX / RGBA	Uso
Fondo general	#111827 → #0f172a	Gradiente de fondo de toda la app
Fondo sidebar	rgba(14,20,38,0.97)	Sidebar izquierdo semi-opaco
Fondo tarjetas	rgba(14,20,38,0.8)	Tablas y paneles de contenido
Azul primario	#3b5bdb	Botones de acción, activos, gradientes
Azul claro	#60a5fa	Links, emails, etiquetas, íconos
Texto principal	#f1f5f9	Encabezados y títulos
Texto secundario	#94a3b8	Textos de apoyo, placeholders
Texto sutil	#3a5070	Labels, metadatos
Borde general	rgba(255,255,255,0.07)	Separadores y bordes de paneles

9.2 Badges de estado

Los campos estado, rol y tipo se renderizan como badges (pastillas de color). El objeto ESTADO_COLORS mapea cada valor posible a tres propiedades: bg (fondo), text (texto) y border (borde). Todos usan RGBA con baja opacidad para un look suave sobre el fondo oscuro.

Valor	Color texto	Significado
PROPUESTO	#94a3b8 (gris)	Proyecto en etapa inicial
ANALISIS / PAUSADO	#fbbf24 (amarillo)	En revisión o pausado temporalmente

Valor	Color texto	Significado
APROBADO / COMPLETADO / ACTIVA / ACTIVO	#34d399 (verde)	Estado positivo o activo
PLANIFICACION	#60a5fa (azul claro)	En planificación
EJECUCION	#3b82f6 (azul medio)	En ejecución activa
SEGUIMIENTO	#a78bfa (violeta)	En seguimiento post-ejecución
CANCELADO / INACTIVA / INACTIVO	#f87171 (rojo)	Estado negativo o inactivo
CERRADO / DEPRECADA	#64748b (gris oscuro)	Estado terminal o abandonado
ADMINISTRADOR	#818cf8 (índigo)	Rol de máximo acceso
ADMINISTRADOR_CLIENTE	#a78bfa (violeta)	Rol intermedio
CLIENTE	#60a5fa (azul)	Rol de usuario final
SIN_CONEXION	#fbbf24 (amarillo)	Dispositivo sin conectividad

9.3 Animaciones CSS

Animación	Aplicada en	Efecto
fadeUp	Login card y logo	Aparece desde abajo con fade in al cargar la pantalla
slideUp	Modales	El modal sube suavemente al abrirse
toastIn	Toast de notificación	El toast entra deslizando desde la derecha
pulse	Indicador API conectada	Punto verde que pulsa para indicar conexión activa

10. Configuración y Variables

El frontend no usa un archivo .env en la configuración actual del proyecto. La URL de la API está definida directamente como constante en App.jsx. Para un entorno de producción, se recomienda migrar esta constante a una variable de entorno de Vite.

10.1 Constante de configuración actual

```
// src/App.jsx — línea 3
const API_BASE = "https://presymphonic-nonnavigably-rhys.ngrok-free.dev";
```

10.2 Migración recomendada a variables de entorno Vite

Para separar la configuración del código, crear un archivo .env en la raíz del proyecto:

```
# .env
VITE_API_BASE=https://tu-dominio-produccion.com
```

Y reemplazar la constante en App.jsx:

```
const API_BASE = import.meta.env.VITE_API_BASE;
```

Nota: Las variables de Vite deben comenzar con el prefijo VITE_ para ser accesibles desde el código del cliente.

11. Integración Frontend ↔ Backend

El frontend y el backend se comunican exclusivamente mediante la API REST. El frontend nunca accede directamente a la base de datos. La siguiente tabla resume la correspondencia entre las entidades del frontend y los endpoints del backend.

Módulo Frontend	Endpoint Backend	ID usado
Clientes	/api/clientes/	id_cliente
Usuarios	/api/usuarios/	id_usuario
Proyectos	/api/proyectos/	id_proyecto
Plantillas	/api/plantillas/	id_plantilla
Archivos	/api/archivos/	id_archivo
Dispositivos	/api/dispositivos/	id_dispositivo

11.1 CORS y configuración de origen

El backend Django tiene configurado CORS_ALLOW_ALL_ORIGINS = True para desarrollo, lo que permite peticiones desde cualquier origen. Los orígenes de confianza explícitos incluyen:

- http://localhost:5173 (servidor de desarrollo Vite)
- http://localhost:3000 (puerto alternativo React)
- https://presymphonic-nonnavigably-rhys.ngrok-free.dev (URL ngrok del backend)

Fin de la documentación